AGENDA CLARA

Sistema de gestión de clientes, servicios y reservas

INFORME DE ENTREGA — VERSIÓN 1

Asignatura	Herramientas de Desarrollo
Grupo	05
Repositorio	github.com/Eduardo-Jara03/Proyecto-pedidos-tiendatecnologia-grupo05
Documento	Adaptación a la consigna oficial de gestión colaborativa

Integrantes registrados

- Eduardo Jesús Jara Moya — U21302190
- Lisbert Wiliams Galvan Zamudio — U22217814
- Nery Emerson Velasquez Pacheco — U23272063
- Carlos Fernando Rivera Hipolito — U23216154
- Bruno Santiago Reyes Vicente — U19221035

Confirmar con el equipo la lista, las responsabilidades y la participación antes de la entrega. La asistencia a la sustentación es obligatoria según la consigna.

1. Descripción de la solución

Agenda Clara es una aplicación web de primera versión para centralizar la administración de clientes, catálogo de servicios y reservas de una empresa de servicios. La interfaz se comunica con una API REST desarrollada con Node.js y Express. La información se persiste en SQLite con relaciones entre clientes, servicios y reservas.

2. Requerimientos y alcance de la V1

Código	Requerimiento	Versión
RF01	Registrar clientes	V1
RF02	Modificar información de clientes	V1
RF03	Consultar y buscar clientes	V1
RF04	Registrar servicios	V1
RF05	Establecer precio y duración de los servicios	V1
RF06	Consultar servicios	V1
RF07	Registrar reservas	V1
RF08	Seleccionar cliente y servicio para la reserva	V1
RF09	Registrar fecha y hora de la reserva	V1
RF10	Consultar reservas	V1
RF11	Cancelar reservas	V1
RF12	Recordatorios automáticos	Versión posterior
RF13	Reportes	Versión posterior
RF14	Dashboard de indicadores	Versión posterior
RF15	Pagos en línea	Versión posterior
RF16	Historial ampliado de clientes	Versión posterior

Los requerimientos y las actividades asociadas también se describen en docs/requerimientos-trazabilidad.md y docs/matriz-trazabilidad.md.

3. Organización del proyecto y repositorio

El proyecto se trabaja en el repositorio GitHub indicado en la carátula. El historial de Git registra cambios del código. Para demostrar la organización, el equipo debe mantener accesibles el repositorio y el tablero, y aportar enlaces a las ramas y Pull Requests que correspondan a esta adaptación.

4. Gestión de actividades

La consigna exige un tablero con cinco estados: Pendiente, Seleccionado, En progreso, En revisión y Hecho. El archivo docs/tablero-actividades.csv sirve como base para cargar tareas en GitHub Projects u otra herramienta colaborativa elegida por el equipo.

EVIDENCIA PENDIENTE: enlace al tablero real y capturas donde se vean los cinco estados, tareas, responsables y evolución. La plantilla CSV no sustituye al tablero publicado.

5. Estrategia de ramas y desarrollo paralelo

- main: versión estable.

- develop: integración de funcionalidades verificadas.
- feature/clientes: cambios del módulo de clientes.
- feature/servicios: cambios del módulo de servicios.
- feature/reservas: cambios del módulo de reservas.
- docs/documentacion: documentación, trazabilidad e informe.

Las ramas deben crearse y publicarse en el repositorio real. Cada cambio se propone mediante Pull Request hacia la rama de integración, con revisión antes de aceptar la integración. Esta lista describe la estrategia propuesta; no afirma que todas las ramas ya estén publicadas.

6. Funcionalidades técnicas implementadas en la adaptación

- API para registrar, consultar/buscar y modificar clientes.
- API para registrar, consultar y modificar servicios con precio y duración; también permite activar o desactivar servicios.
- API para registrar, consultar/buscar y cancelar reservas vinculadas a un cliente y un servicio, con fecha y hora.
- Persistencia SQLite y validaciones básicas de datos.
- Pruebas automatizadas con Node.js Test Runner.

Rutas principales: /api/clientes, /api/servicios, /api/reservas y /api/health.

7. Pruebas y validación técnica

En el entorno de preparación se ejecutó npm test: 20 pruebas aprobadas y 0 fallidas. Las pruebas cubren estado de API/SQLite, registro y modificación de clientes, validaciones, servicios, registro y consulta de reservas, asociación entre entidades y cancelación.

PENDIENTE DE VALIDACIÓN POR EL EQUIPO: ejecutar npm test en el Windows usado para la entrega, conservar la salida de consola y probar manualmente los flujos principales desde el navegador. El resultado del entorno de preparación no debe presentarse como captura obtenida en el equipo del estudiante.

8. Trabajo colaborativo y responsabilidades

Los integrantes y las responsabilidades deben comprobarse usando commits, asignaciones del tablero y Pull Requests. No se deben atribuir tareas, revisiones o aprobaciones a una persona sin evidencia verificable.

EVIDENCIAS PENDIENTES: tareas asignadas, contribuciones por integrante, commits relevantes y enlaces a PR con revisión auténtica de un compañero.

9. Integración y revisión de cambios

- El responsable desarrolla en su rama de funcionalidad.
- Ejecuta las pruebas correspondientes.
- Abre una Pull Request hacia develop.
- Otro integrante revisa el cambio y registra observaciones.
- El autor corrige lo necesario.
- Se integra tras la revisión y la validación.

EVIDENCIA PENDIENTE: enlace a una Pull Request real que muestre revisión previa a la aceptación. La descripción del procedimiento no sustituye esa evidencia.

10. Gestión de conflictos

La evaluación solicita al menos un conflicto originado por trabajo simultáneo. Debe reproducirse de forma real, resolverse y documentarse con el origen, archivos involucrados, decisión adoptada y resultado final.

El archivo docs/conflicto-git-plantilla.md es una plantilla de registro, no prueba de que un conflicto ya haya ocurrido. Debe completarse después de una resolución real y adjuntar evidencia de Git o captura.

11. Gestión de versiones y liberación

La V1 prevista incluye gestión de clientes, servicios y reservas. Se excluyen de esta versión los recordatorios automáticos, reportes, dashboard, pagos en línea e historial ampliado. La versión estable deberá identificarse con un tag Git (por ejemplo, v1.0.0) únicamente después de integrar y verificar las funcionalidades.

No se afirma en este informe que la etiqueta o la liberación final ya exista.

12. Trazabilidad

La correspondencia exigida es: Requerimiento → Actividad → Responsable → Desarrollo → Revisión → Integración. La matriz debe incluir para cada requisito su código, tarea, responsable real, rama/commit o archivo asociado, PR/revisor y evidencia de integración. Los campos sin evidencia deben completarse antes de la entrega.

13. Lista de verificación final

- Confirmar nombres, códigos y participación de integrantes.
- Publicar las ramas acordadas para la adaptación.
- Crear y compartir el tablero con los cinco estados.
- Completar responsables y prioridades de actividades.
- Ejecutar npm test en Windows y guardar evidencia.
- Probar manualmente clientes, servicios y reservas en navegador.
- Adjuntar una Pull Request con revisión auténtica previa a la integración.
- Reproducir, resolver y documentar un conflicto real.
- Integrar y verificar la V1.
- Crear el tag de versión estable después de validar.
- Incorporar capturas y enlaces reales en el PDF definitivo.
- Asegurar asistencia a la sustentación.

14. Conclusiones

La adaptación de Agenda Clara orienta el código a los requerimientos funcionales de clientes, servicios y reservas de la consigna. En el entorno de preparación, las pruebas automatizadas registraron 20 aprobaciones y ninguna falla. Sin embargo, la evaluación también depende de evidencias reales de organización, tablero, colaboración, revisión, resolución de conflicto, integración y versión liberada. Este informe distingue entre lo implementado técnicamente y las evidencias que el equipo debe completar antes de declarar la entrega final.